МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут ІКНІ
Кафедра СШІ

ЗВІТ

до лабораторної роботи № 4
з дисципліни: «Інтелектуальний аналіз та візуалізація даних»
на тему: «Візуальний аналіз даних та часові ряди: той самий аналіз у двох середовищах»

Виконав: ст. гр. ШІ-33 Левченко Д.К.
Прийняла: Торська Роксана

Львів – 2026

Тема роботи:

Візуальний аналіз даних та часові ряди: повний цикл аналізу одного часового ряду засобами Python і засобами Orange та пояснення, чому однаково названі метрики якості прогнозу дають у двох середовищах різні числа.

Мета роботи:

Навчитися готувати часовий ряд до аналізу (повна місячна сітка, заповнення пропуску), будувати графіки з обґрунтуванням геометрії та шкали, розкладати ряд на тренд, сезонну складову й залишок, перевіряти стаціонарність парою тестів ADF і KPSS, оцінювати прогнози ретроспективною перевіркою з рухомим початком, порівнювати модель ARIMA з наївними базисами, відтворювати той самий аналіз в Orange і пояснювати розбіжність метрик через їхні означення.

Індивідуальне завдання (варіант 4):

Варіант визначено службовою клітинкою зошита за номером у списку групи (4): ряд — роздрібна ціна сметани на обласному ринку Херсона (табл. 1).

Таблиця 1 — Індивідуальне завдання

Параметр	Значення
Варіант	4 (номер у списку групи — 4)
Ринок	Kherson (Херсонська область)
Товар	Sour cream – Retail (сметана), грн за літр
Дані	ціни_україна_2014_2017.csv — World Food Programme, Global Food Prices Database, ліцензія CC BY-IGO
Період	03.2014 – 12.2017, 46 місяців
Перевірка	7 фолдів по 3 кроки, рухомий початок, ARIMA(1, 1, 1)
Середовища	Python 3.11 (pandas, statsmodels) і Orange 3.40 з додатком Orange3-Timeseries 0.6.3

ЗАВДАННЯ

1. Витягти ряд свого варіанта і привести його до повної місячної сітки, знайти й пояснити пропуск.
2. Заповнити пропуск трьома способами й обґрунтувати вибір.
3. Побудувати щонайменше три графіки, що відповідають на різні питання, з поясненням геометрії та шкали.
4. Розкласти ряд методом STL з періодом 12 і обчислити силу сезонності та тренду.

5. Побудувати ACF і PACF, перевірити стаціонарність тестами ADF і KPSS до і після диференціювання.
6. Побудувати чотири базисні прогнози й оцінити їх ретроспективною перевіркою з рухомим початком (7 × 3).
7. Побудувати й оцінити модель ARIMA(1, 1, 1) і порівняти її з найкращим базисом.
8. Знайти точку зміни режиму скануванням і перевірити її значущість.
9. Відрецензувати чужий аналіз і знайти методологічні помилки.
10. Експортувати ряд для Orange.
11. Побудувати в Orange потік File → As Timeseries → Interpolate → Line Chart.
12. Додати Correlogram, Difference і Seasonal Adjustment та порівняти з Python.
13. Додати ARIMA Model і Model Evaluation (7 фолдів × 3 кроки) та пояснити відмову при 20 фолдах.
14. Відтворити в Python кожну з чотирьох метрик Orange і пояснити розбіжність.
15. Розгорнути метрику POCID у матрицю напрямку зміни 2 × 2.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Часовий ряд — послідовність вимірів, упорядкованих у часі з рівним кроком; у цій роботі крок — місяць. Перш ніж аналізувати ряд, його приводять до повної сітки дат: пропущений місяць має з’явитися як порожнє значення, інакше всі зсуви, лаги й сезонний період після пропуску зміщуються.

Метод STL (Seasonal-Trend decomposition using Loess) розкладає ряд на тренд T, сезонну складову S і залишок R. STL повертає сезонну складову завжди, тому її значущість оцінюють силою сезонності $F_S = \max(0;\ 1 - \mathrm{Var}(R) / \mathrm{Var}(S + R))$; значення нижче 0,3 означає, що сезонності фактично немає. Аналогічно сила тренду $F_T = \max(0;\ 1 - \mathrm{Var}(R) / \mathrm{Var}(T + R))$.

Стаціонарність перевіряють парою тестів із протилежними нульовими гіпотезами: у тесті ADF H_0 — ряд нестаціонарний (мале p означає стаціонарність), у тесті KPSS H_0 — ряд стаціонарний (мале p означає нестаціонарність). Якщо тести суперечать один одному, це змістовний результат, а не помилка обчислень.

Якість прогнозу оцінюють ретроспективною перевіркою з рухомим початком: модель навчається на перших k спостереженнях, прогнозує наступні h, початок

зсувається. У роботі жорстко задано 7 фолдів по 3 кроки, відлічуваних від кінця ряду: для фолду f навчання — усі місяці до позиції −3·f. Будь-яку модель порівнюють з наївними базисами: наївним (остання ціна), сезонним наївним (ціна рік тому), середнім і дрейфом (продовження середнього нахилу). Метрики: MAE = mean|e|, RMSE = √(mean e^2), MAPE = mean|e / y|·100 %.

Додаток Orange3-Timeseries обгортає ту саму statsmodels.tsa.arima.model.ARIMA, тож при однакових даних, моделі й фолдах прогнози збігаються до знака. Розбіжність у таблиці Orange походить лише з означень метрик (табл. 2).

Таблиця 2 — Метрики в Orange і їхні звичні означення

Метрика	Що рахує Orange	Що зазвичай так називають	Наслідок
RMSE	$\sqrt{(\Sigma e^2 / n)}$	те саме	збігається повністю
MAE	median |e|	mean |e|	різні величини
MAPE	mean|e| / mean|y|	mean |e / y| · 100 %	інша формула, і це частка, а не відсоток
POCID	100 · частка пар зі збігом напрямку зміни	у scikit-learn немає	міряє напрямок, а не величину похибки

ХІД РОБОТИ

Етап 0. Варіант

У службову клітинку вписано прізвище, групу й номер 4; друга службова клітинка зчитала lr4_variants_student.json і визначила змінні MARKET = Kherson і GOOD = Sour cream – Retail, з якими працює весь подальший код.

Результат виконання:

```
======================================================================
ВАРІАНТ 4   ·   Левченко Денис   ·   ШІ-33
======================================================================


   Ринок:  Kherson
   Товар:  сметана   (у наборі даних: Sour cream - Retail)


Далі в роботі користуйтеся змінними MARKET і GOOD —
тоді весь код автоматично працюватиме саме з вашим рядом.


Період: 03.2014 – 12.2017 · очікувана довжина ряду: 46 місяців
Фолдів ретроспективної перевірки: 7 · горизонт: 3 місяці · ARIMA(1, 1, 1)
```

Етап 1. Підготовка ряду

```
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Image, display


plt.rcParams['figure.dpi'] = 110
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3


prices = pd.read_csv('ціни_україна_2014_2017.csv')
prices['date'] = pd.to_datetime(dict(year=prices['рік'], month=prices['місяць'],
day=1))
raw = (prices[(prices['ринок'] == MARKET) & (prices['товар'] == GOOD)]
       .set_index('date')['ціна'].sort_index())
unit = prices.loc[(prices['ринок'] == MARKET) & (prices['товар'] == GOOD),
['одиниця', 'валюта']].iloc[0]
print('ряд:', MARKET, '|', GOOD, '| одиниця:', unit['одиниця'], '| валюта:',
unit['валюта'])


grid = pd.date_range('2014-03-01', '2017-12-01', freq='MS')
s = raw.reindex(grid)
print('місяців у сітці     :', len(grid))
print('мають вимір          :', int(s.notna().sum()))
print('пропущені місяці     :', [d.strftime('%m.%Y') for d in s[s.isna()].index])


per_month = prices.groupby('date').size().reindex(grid, fill_value=0)
print('записів у всьому наборі за 08.2017:', int(per_month['2017-08-01']))
print('місяці без жодного запису в наборі:', [d.strftime('%m.%Y') for d in
per_month[per_month == 0].index])
```

Результат виконання:

```
ряд: Kherson | Sour cream - Retail | одиниця: L | валюта: UAH
місяців у сітці     : 46
мають вимір          : 45
пропущені місяці     : ['08.2017']
записів у всьому наборі за 08.2017: 0
місяці без жодного запису в наборі: ['08.2017']
```

Повна місячна сітка 03.2014–12.2017 має 46 місяців, вимір є для 45. Пропущено серпень 2017 року. Пропуск не пов’язаний із ринком Херсона: у всьому наборі за 08.2017 немає жодного запису для жодного ринку й товару, тобто джерело не опублікувало цей місяць. Тому пропуск у ряді один і такий самий в усіх рядах набору.

Етап 2. Заповнення пропуску

```
variants = pd.DataFrame({
    'перенесення останнього': s.ffill(),
    'лінійна інтерполяція': s.interpolate('linear'),
    'середнє 4 сусідів': s.fillna(s.rolling(5, center=True, min_periods=1).mean()),
})
print('значення за 08.2017:')
```

```
print(variants.loc['2017-08-01'].round(4).to_string())
print()
print('сусіди: 07.2017 = %.4f, 09.2017 = %.4f' % (s['2017-07-01'], s['2017-09-01']))


y = s.interpolate('linear')
print('обрано лінійну інтерполяцію: 08.2017 =', round(y['2017-08-01'], 4))


coverage = prices.groupby('ринок')['date'].agg(['min', 'max', 'nunique'])
print()
print('ринки з неповним рядом:')
print(coverage[coverage['nunique'] < 45].to_string())
```

Результат виконання:

```
значення за 08.2017:
перенесення останнього    44.5400
лінійна інтерполяція      44.3734
середнє 4 сусідів         44.5900


сусіди: 07.2017 = 44.5400, 09.2017 = 44.2067
обрано лінійну інтерполяцію: 08.2017 = 44.3734


ринки з неповним рядом:
                     min        max  nunique
ринок
Crimea        2014-03-01 2014-03-01        1
Donetska      2014-03-01 2017-12-01       42
Luhanska      2014-03-01 2017-12-01       40
m. Sevastopol 2014-03-01 2014-04-01        2
```

Порівняно три способи: перенесення останнього значення дає 44,54 грн, лінійна інтерполяція — 44,37 грн, середнє чотирьох сусідів (06, 07, 09 і 10.2017) — 44,59 грн. Обрано **лінійну інтерполяцію**: пропуск один, між двома відомими місяцями, а ціна за ці два місяці майже не змінилась (−0,33 грн), тож середина між сусідами — найменш довільне припущення. Перенесення останнього штучно робить у серпні нульову зміну, а середнє чотирьох сусідів захоплює жовтневий стрибок і дає значення вище за обох найближчих сусідів. До того ж у Orange той самий тип інтерполяції (Linear interpolation), тож обидва середовища отримують однаковий ряд.

Заповнювати неприпустимо, коли відсутність даних означає, що спостереження припинилось. У цьому наборі Крим має лише 03.2014, Севастополь — 03–04.2014, у Донецькій області немає 08, 09 і 12.2014, а в Луганській — 08–12.2014. «Заповнити» Крим на 2014–2017 означало б вигадати 44 місяці цін, яких ніхто не вимірював, — це вже фабрикація даних, а не оцінка одного пропущеного виміру.

Етап 3. Візуальна розвідка

```
filled = y.index == pd.Timestamp('2017-08-01')


fig, ax = plt.subplots(figsize=(11, 3.8))
ax.plot(y.index, y.values, color='#2C6E9B', lw=2, marker='o', ms=3)
ax.scatter(y.index[filled], y[filled], s=70, facecolor='white', edgecolor='#C0392B',
lw=2, zorder=5,
           label='заповнений пропуск 08.2017')
ax.set_title('1. Як змінювалась ціна сметани в Херсоні, грн/л')
ax.set_ylabel('грн за літр')
ax.legend()
plt.tight_layout()
plt.show()


mom = y.pct_change() * 100
fig, ax = plt.subplots(figsize=(11, 3.4))
ax.bar(mom.index, mom.values, width=20, color=np.where(mom > 0, '#C0392B',
'#2C6E9B'))
ax.axhline(0, color='black', lw=0.8)
ax.set_title('2. Коли ціна стрибала: зміна до попереднього місяця, %')
ax.set_ylabel('% за місяць')
plt.tight_layout()
plt.show()


seas = pd.DataFrame({'рік': mom.index.year, 'місяць': mom.index.month, 'зміна':
mom.values}).dropna()
fig, ax = plt.subplots(figsize=(11, 3.6))
for year, g in seas.groupby('рік'):
    ax.plot(g['місяць'], g['зміна'], marker='o', lw=1, alpha=0.8, label=str(year))
ax.plot(range(1, 13), seas.groupby('місяць')['зміна'].mean().reindex(range(1, 13)),
color='black', lw=3,
        label='середнє за роки')
ax.axhline(0, color='black', lw=0.8)
ax.set_xticks(range(1, 13))
ax.set_xticklabels(['січ', 'лют', 'бер', 'кві', 'тра', 'чер', 'лип', 'сер', 'вер',
'жов', 'лис', 'гру'])
ax.set_title('3. Чи повторюється річний рисунок: місячні зміни, накладені за
календарем')
ax.set_ylabel('% за місяць')
ax.legend(ncol=5)
plt.tight_layout()
plt.show()


fig, ax = plt.subplots(figsize=(11, 3.4))
ax.semilogy(y.index, y.values, color='#2C6E9B', lw=2)
ax.set_title('4. Логарифмічна шкала: чи стала швидкість зростання')
ax.set_ylabel('грн за літр, лог. шкала')
plt.tight_layout()
plt.show()


print('найбільші місячні зростання, %:', {d.strftime('%m.%Y'): round(v, 2) for d, v
in mom.nlargest(5).items()})
print('місяців зі зниженням ціни:', int((mom < 0).sum()), '| найбільше зниження:',
mom.idxmin().strftime('%m.%Y'),
      round(mom.min(), 2), '%')
year_growth = y.groupby(y.index.year).agg(['first', 'last'])
print('зростання за рік, разів:', (year_growth['last'] /
year_growth['first']).round(3).to_dict())
winter = mom[mom.index.month.isin([12, 1, 2, 3])].mean()
summer = mom[mom.index.month.isin([5, 6, 7, 8, 9])].mean()
print('середня зміна в грудні–березні: %.2f %% | у травні–вересні: %.2f %%' %
(winter, summer))
```



Рисунок 1 — Ціна сметани в Херсоні, грн за літр (графік 1)



Рисунок 2 — Зміна ціни до попереднього місяця, % (графік 2)

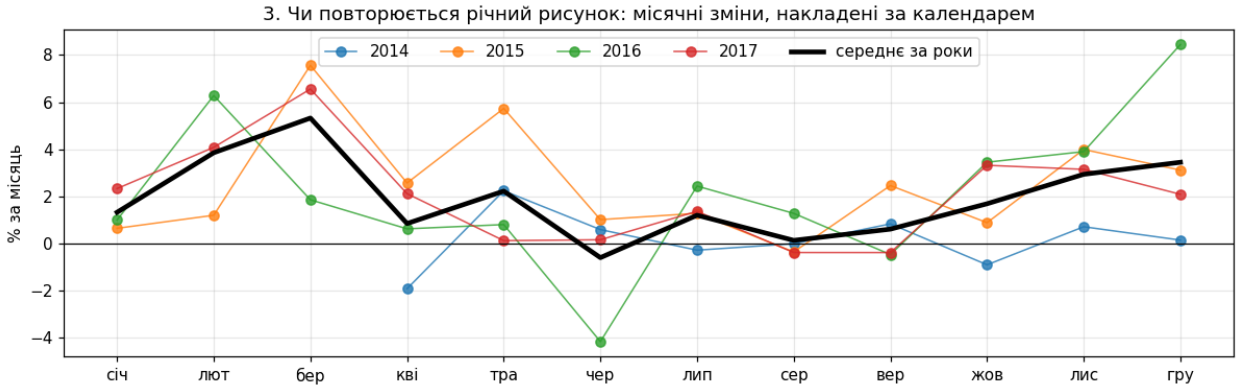


Рисунок 3 — Місячні зміни, накладені за календарним місяцем (графік 3)



Рисунок 4 — Ціна в логарифмічній шкалі (графік 4)

Результат виконання:

```
найбільші місячні зростання, %: {'12.2016': 8.45, '03.2015': 7.56, '03.2017': 6.56,
'02.2016': 6.28, '05.2015': 5.74}
місяців зі зниженням ціни: 8 | найбільше зниження: 06.2016 -4.15 %
зростання за рік, разів: {2014: 1.014, 2015: 1.335, 2016: 1.267, 2017: 1.244}
середня зміна в грудні–березні: 3.49 % | у травні–вересні: 0.72 %
```

Кожен графік ставить своє питання. Рис. 1 — як змінювалась ціна: лінія, бо точки впорядковані в часі; вісь Y не від нуля, бо питання про зміну, а лінія кодує величину положенням, а не довжиною; заповнений місяць позначено окремим маркером. Рис. 2 — коли були стрибки: стовпчики кодують величину довжиною, тому відлік від нуля обов'язковий, а відсотки замість гривень порівнюють зміни 2014 і 2017 років на одній шкалі. Рис. 3 — чи повторюється рисунок усередині року: вісь X — календарний місяць, беруться зміни, а не рівень, інакше тренд розвів би роки по вертикалі. Рис. 4 — чи стала швидкість зростання: на логарифмічній шкалі однаковий відсоток дає однаковий нахил.

Несподівана знахідка: найбільший місячний стрибок стався не у 2015 році, а в грудні 2016 (+8,45 %), а зростання зосереджене в грудні–березні (у середньому +3,49 % за місяць проти +0,72 % у травні–вересні). Ціна знижувалась лише 8 разів із 45, найбільше — на 4,15 % у червні 2016.

Етап 4. Розкладання на складові

```
from statsmodels.tsa.seasonal import STL


stl = STL(y, period=12).fit()
fig = stl.plot()
fig.set_size_inches(11, 7)
fig.suptitle('STL, період 12: ряд, тренд, сезонна складова, залишок', y=1.0)
plt.tight_layout()
plt.show()


F_S = max(0, 1 - stl.resid.var() / (stl.seasonal + stl.resid).var())
F_T = max(0, 1 - stl.resid.var() / (stl.trend + stl.resid).var())
print('сила сезонності F_S = %.3f' % F_S)
print('сила тренду     F_T = %.3f' % F_T)
print('амплітуда сезонної складової: %.2f грн' % (stl.seasonal.max() -
stl.seasonal.min()))


stl_r = STL(y, period=12, robust=True).fit()
print('перевірка робастним STL: F_S = %.3f' % max(0, 1 - stl_r.resid.var() /
(stl_r.seasonal + stl_r.resid).var()))
by_month = stl.seasonal.groupby(stl.seasonal.index.month).mean()
print('сезонна складова за календарними місяцями, грн:')
print(by_month.round(2).to_dict())
```

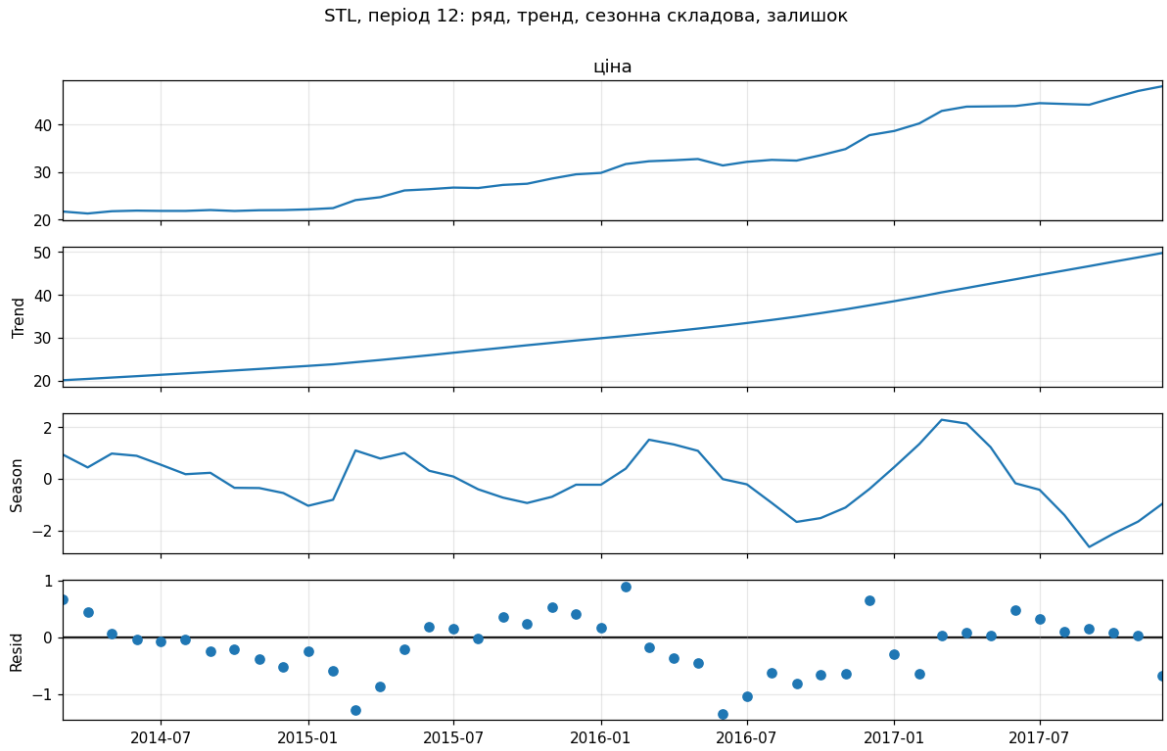


Рисунок 5 — Розкладання STL з періодом 12: ряд, тренд, сезонна складова, залишок

Результат виконання:

```
сила сезонності F_S = 0.823
сила тренду     F_T = 0.997
амплітуда сезонної складової: 4.92 грн
перевірка робастним STL: F_S = 0.525
сезонна складова за календарними місяцями, грн:
{1: -0.28, 2: 0.3, 3: 1.45, 4: 1.17, 5: 1.06, 6: 0.25, 7: -0.01, 8: -0.65, 9: -1.21,
10: -1.24, 11: -0.96, 12: -0.54}
```

Сила сезонності F_S = 0,823, сила тренду F_T = 0,997; робастний STL дає F_S = 0,525. **Сезонність у ряді є**: обидві оцінки вищі за поріг 0,3. Сезонна складова найвища в березні–травні (+1,1…+1,5 грн) і найнижча у вересні–жовтні (−1,2 грн): ціна підтягується взимку–навесні й стоїть влітку–восени, що узгоджується з графіком 3 і сезонністю виробництва молока. Проте тренд значно сильніший: амплітуда сезонної складової 4,9 грн проти зростання на 26 грн за період, а на 46 точках (менше чотирьох повних циклів) оцінка сезонності нестійка.

Етап 5. Стаціонарність

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller, kpss


warnings.filterwarnings('ignore')
dy = y.diff().dropna()
```

```
fig, axes = plt.subplots(2, 2, figsize=(11, 6))
plot_acf(y, lags=15, ax=axes[0, 0], title='ACF рівня')
plot_pacf(y, lags=15, ax=axes[0, 1], method='ywm', title='PACF рівня')
plot_acf(dy, lags=15, ax=axes[1, 0], title='ACF першої різниці')
plot_pacf(dy, lags=15, ax=axes[1, 1], method='ywm', title='PACF першої різниці')
plt.tight_layout()
plt.show()


rows = []
for name, x in [('рівень', y), ('перша різниця', dy)]:
    rows.append({'ряд': name,
                 'ADF p (H0: нестаціонарний)': adfuller(x, autolag='AIC')[1],
                 'KPSS p (H0: стаціонарний)': kpss(x, regression='c',
nlags='auto')[1]})
tests = pd.DataFrame(rows).set_index('ряд')
display(tests.round(4))


from statsmodels.tsa.stattools import acf, pacf
print('межа значущості ±%.3f (n = %d)' % (1.96 / np.sqrt(len(dy)), len(dy)))
print('ACF різниці, лаги 1–12 :', np.round(acf(dy, nlags=12)[1:], 2))
print('PACF різниці, лаги 1–12:', np.round(pacf(dy, nlags=12, method='ywm')[1:], 2))
```

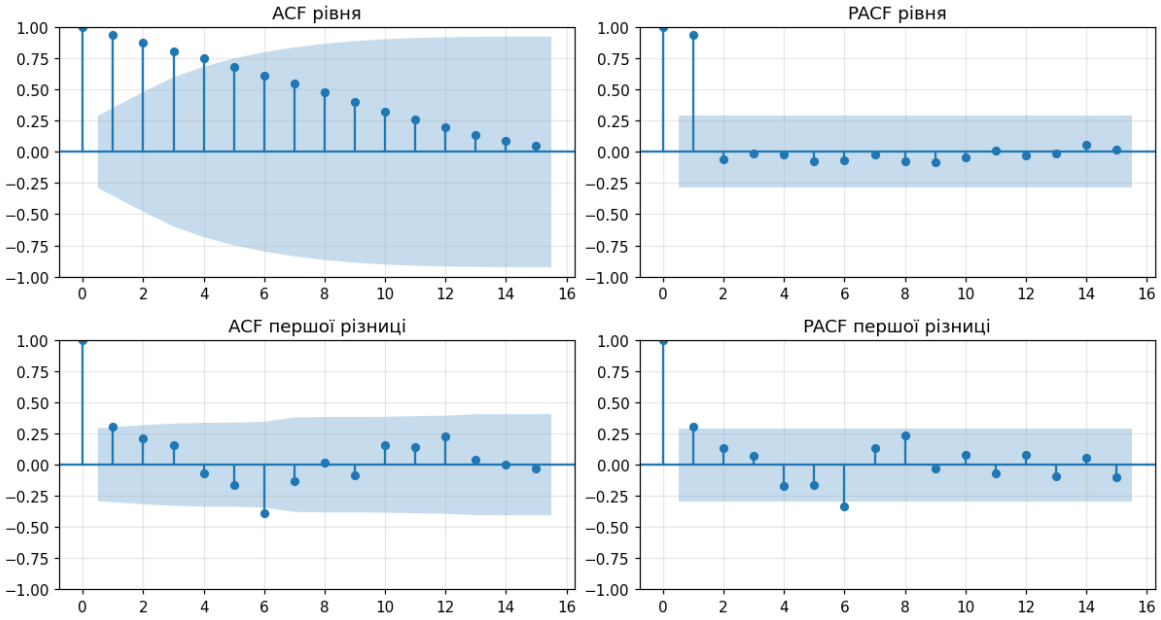


Рисунок 6 — ACF і PACF рівня та першої різниці

Таблиця 3 — p-значення тестів стаціонарності

Ряд	ADF p (H_0: нестаціонарний)	KPSS p (H_0: стаціонарний)	Висновок
рівень	0,9991	≤ 0,01	нестаціонарний
перша різниця	0,0015	≥ 0,10	стаціонарний

Результат виконання:

```
межа значущості ±0.292 (n = 45)
ACF різниці, лаги 1–12 : [ 0.3   0.21  0.16 -0.07 -0.16 -0.39 -0.13  0.02 -0.08
0.15  0.14  0.23]
PACF різниці, лаги 1–12: [ 0.3   0.13  0.07 -0.17 -0.16 -0.34  0.13  0.24 -0.03
0.08 -0.07  0.08]
```

Рівень нестаціонарний: ADF не відхиляє H_0 (p = 0,999), KPSS відхиляє H_0 (p ≤ 0,01). Перша різниця стаціонарна: ADF відхиляє H_0 (p = 0,0015), KPSS не відхиляє (p ≥ 0,10). Тести узгоджуються для обох рядів, тож порядок диференціювання d = 1. ACF рівня згасає повільно (0,94 → 0,20 на лагу 12) — ознака тренду; у різниці лаг 1 = 0,30 ледь виходить за межу ±0,29, а значущий від'ємний лаг 6 (−0,39) показує пів-річний рисунок.

Етап 6. Базисні прогнози та ретроспективна перевірка

```
FOLDS, H = 7, 3
values = y.values
n = len(values)


def folds():
    for f in range(1, FOLDS + 1):
        yield f, values[:n - H * f], values[n - H * f:n - H * f + H]


def forecast(method, train, h):
    if method == 'наївний':
        return np.repeat(train[-1], h)
    if method == 'сезонний наївний':
        return np.array([train[len(train) - 12 + i] for i in range(h)])
    if method == 'середнє':
        return np.repeat(train.mean(), h)
    if method == 'дрейф':
        slope = (train[-1] - train[0]) / (len(train) - 1)
        return train[-1] + slope * np.arange(1, h + 1)


def scores(true, pred):
    e = true - pred
    return {'MAE': np.mean(np.abs(e)), 'RMSE': np.sqrt(np.mean(e ** 2)), 'MAPE, %':
np.mean(np.abs(e / true)) * 100}


BASELINES = ['наївний', 'сезонний наївний', 'середнє', 'дрейф']
cv = {}
for m in BASELINES:
    true = np.concatenate([te for _, _, te in folds()])
    pred = np.concatenate([forecast(m, tr, H) for _, tr, _ in folds()])
    cv[m] = scores(true, pred)
cv = pd.DataFrame(cv).T
display(cv.round(3))


print('фолди: навчання до позиції −3·f, перевірка наступних 3 місяців')
for f, tr, te in folds():
    print('  фолд %d: навчання 03.2014–%s (%d міс.) → перевірка %s–%s'
          % (f, y.index[len(tr) - 1].strftime('%m.%Y'), len(tr),
y.index[len(tr)].strftime('%m.%Y'),
             y.index[len(tr) + H - 1].strftime('%m.%Y')))


from sklearn.model_selection import train_test_split


idx_tr, idx_te = train_test_split(np.arange(n), test_size=0.25, random_state=42)
idx_tr, idx_te = np.sort(idx_tr), np.sort(idx_te)
rnd = {}
for m in ['наївний', 'середнє', 'дрейф']:
    pred = []
```

```
    for t in idx_te:
        before = idx_tr[idx_tr < t]
        if m == 'наївний':
            pred.append(values[before[-1]] if len(before) else values[idx_tr[0]])
        elif m == 'середнє':
            pred.append(values[idx_tr].mean())
        else:
            slope = (values[idx_tr[-1]] - values[idx_tr[0]]) / (idx_tr[-1] -
idx_tr[0])
            pred.append(values[idx_tr[0]] + slope * (t - idx_tr[0]))
    rnd[m] = np.mean(np.abs(values[idx_te] - np.array(pred)))
compare = pd.DataFrame({'MAE рухомий початок': cv.loc[list(rnd), 'MAE'], 'MAE
випадковий 75/25': pd.Series(rnd)})
compare['у скільки разів'] = compare['MAE випадковий 75/25'] / compare['MAE рухомий
початок']
display(compare.round(3))
print('тестові місяці випадкового поділу:', [y.index[i].strftime('%m.%y') for i in
idx_te])
```

Таблиця 4 — Базисні прогнози: 7 фолдів × 3 кроки

Базис	MAE	RMSE	MAPE, %
наївний	1,639	2,207	4,09
сезонний наївний	9,054	9,445	22,50
середнє	11,920	12,570	29,39
дрейф	1,171	1,631	2,96

Результат виконання:

```
фолди: навчання до позиції −3·f, перевірка наступних 3 місяців
  фолд 1: навчання 03.2014–09.2017 (43 міс.) → перевірка 10.2017–12.2017
  фолд 2: навчання 03.2014–06.2017 (40 міс.) → перевірка 07.2017–09.2017
  фолд 3: навчання 03.2014–03.2017 (37 міс.) → перевірка 04.2017–06.2017
  фолд 4: навчання 03.2014–12.2016 (34 міс.) → перевірка 01.2017–03.2017
  фолд 5: навчання 03.2014–09.2016 (31 міс.) → перевірка 10.2016–12.2016
  фолд 6: навчання 03.2014–06.2016 (28 міс.) → перевірка 07.2016–09.2016
  фолд 7: навчання 03.2014–03.2016 (25 міс.) → перевірка 04.2016–06.2016
тестові місяці випадкового поділу: ['06.14', '07.14', '09.14', '11.14', '03.15',
'03.16', '04.16', '05.16', '02.17', '06.17', '08.17', '11.17']
```

Таблиця 5 — Рухомий початок проти випадкового поділу 75/25 (MAE)

Базис	Рухомий початок	Випадковий 75/25	Відношення
наївний	1,639	0,661	0,40
середнє	11,920	8,235	0,69
дрейф	1,171	2,666	2,28

Найкращий базис — **дрейф** (MAE 1,171, RMSE 1,631, MAPE 2,96 %): ряд має сильний майже лінійний тренд (у середньому +0,59 грн за місяць), наївний прогноз застигає на останньому значенні, а дрейф продовжує нахил. Середнє й сезонний наївний найгірші, бо за рік ціна зростає на 25–34 %. Випадковий поділ 75/25 міряє інше: для наївного прогнозу помилка в 2,5 раза менша, бо тестовий місяць майже завжди має відомого сусіда (витік майбутнього), а для дрейфу —

у 2,28 раза більша, бо пряма через перший і останній навчальні місяці не описує вигнутий тренд усередині періоду.

Етап 7. Модель ARIMA

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox


warnings.filterwarnings('ignore')


def arima_cv(order, trend='n'):
    true, pred = [], []
    for _, tr, te in folds():
        fit = ARIMA(tr, order=order, trend=trend).fit()
        true.append(te)
        pred.append(fit.get_forecast(H).predicted_mean)
    return np.concatenate(true), np.concatenate(pred)


arima_true, arima_pred = arima_cv((1, 1, 1))
table = cv.copy()
table.loc['ARIMA(1,1,1)'] = scores(arima_true, arima_pred)
for order in [(1, 1, 0), (0, 1, 1)]:
    t, p = arima_cv(order, trend='t')
    table.loc['ARIMA%s + дрейф' % str(order).replace(' ', '')] = scores(t, p)
display(table.round(3))


best = cv['MAE'].idxmin()
print('найкращий базис: %s' % best)
print('ARIMA(1,1,1) гірша за нього: MAE у %.2f раза, RMSE у %.2f раза'
      % (table.loc['ARIMA(1,1,1)', 'MAE'] / cv.loc[best, 'MAE'],
table.loc['ARIMA(1,1,1)', 'RMSE'] / cv.loc[best, 'RMSE']))


full = ARIMA(values, order=(1, 1, 1)).fit()
print()
print(full.summary().tables[1])
resid = pd.Series(full.resid[1:], index=y.index[1:])
fig, axes = plt.subplots(1, 2, figsize=(11, 3.4))
axes[0].plot(resid.index, resid.values, color='#2C6E9B', marker='o', ms=3)
axes[0].axhline(0, color='black', lw=0.8)
axes[0].set_title('Залишки ARIMA(1,1,1), грн')
plot_acf(resid, lags=15, ax=axes[1], title='ACF залишків')
plt.tight_layout()
plt.show()
print('середнє залишків: %.3f грн (модель без дрейфу)' % resid.mean())
display(acorr_ljungbox(resid, lags=[6, 12], return_df=True).round(4))
```

Таблиця 6 — ARIMA і базиси на тих самих фолдах

Модель	MAE	RMSE	MAPE, %
наївний	1,639	2,207	4,09
сезонний наївний	9,054	9,445	22,50
середнє	11,920	12,570	29,39
дрейф	1,171	1,631	2,96
ARIMA(1,1,1)	1,710	2,174	4,40
ARIMA(1,1,0) + дрейф	1,272	1,646	3,22
ARIMA(0,1,1) + дрейф	1,187	1,618	3,01

Результат виконання:

```
найкращий базис: дрейф
ARIMA(1,1,1) гірша за нього: MAE у 1.46 раза, RMSE у 1.33 раза


==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.8593      0.120      7.147      0.000       0.624       1.095
ma.L1         -0.4714      0.207     -2.273      0.023      -0.878      -0.065
sigma2         0.6095      0.131      4.652      0.000       0.353       0.866
==============================================================================
середнє залишків: 0.191 грн (модель без дрейфу)
```

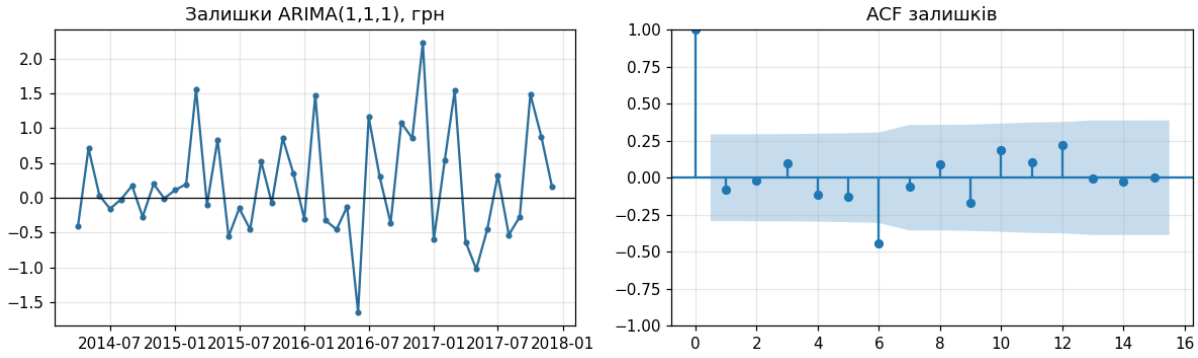


Рисунок 7 — Залишки ARIMA(1, 1, 1) та їхня ACF

ARIMA(1, 1, 1) на 7 фолдах дає MAE 1,710, RMSE 2,174, MAPE 4,40 % і **програє найкращому базису**: MAE у 1,46 раза, RMSE у 1,33 раза гірші, ніж у дрейфу, а за MAE модель поступається навіть наївному прогнозу. Причина — у statsmodels модель з d = 1 не має константи й не бачить дрейфу. За ACF і PACF різниці (обриваються після лагу 1, середня різниця не нуль) обрано б ARIMA(0, 1, 1) з дрейфом: MAE 1,187 і RMSE 1,618 — практично як дрейф. Висновок прямо: ARIMA(1, 1, 1) у такому вигляді права на існування для цього ряду не має. Залишки зміщені (середнє +0,19 грн), а тест Льюнга–Бокса дає p = 0,043 на лагу 6 і p = 0,048 на лагу 12 — структура в залишках лишилась.

Етап 8. Точка зміни режиму

```
def scan(x, min_size=6):
    best_k, best_t = None, 0.0
    for k in range(min_size, len(x) - min_size + 1):
        a, b = x[:k], x[k:]
        t = (b.mean() - a.mean()) / np.sqrt(a.var(ddof=1) / len(a) + b.var(ddof=1) /
len(b))
        if abs(t) > abs(best_t):
            best_k, best_t = k, t
    return best_k, best_t


k, t_stat = scan(values)
print('точка зміни: %s, t = %.2f' % (y.index[k].strftime('%m.%Y'), t_stat))
```

```
print('середнє до: %.2f грн, після: %.2f грн' % (values[:k].mean(),
values[k:].mean()))


rng = np.random.default_rng(42)
steps = np.diff(values)
sim = []
for _ in range(2000):
    fake = values[0] + np.concatenate([[0], np.cumsum(rng.choice(steps,
size=len(steps), replace=True))])
    sim.append(abs(scan(fake)[1]))
sim = np.array(sim)
p_boot = np.mean(sim >= abs(t_stat))
print('бутстреп приростів (2000 рядів без зламу): медіана max|t| = %.1f, 95-й
процентиль = %.1f'
      % (np.median(sim), np.percentile(sim, 95)))
print('p-значення зламу проти «просто тренду» = %.3f' % p_boot)


kd, td = scan(steps)
print('скан приростів: швидкість зростання змінилась у %s, t = %.2f, приріст до
%.2f, після %.2f грн/міс'
      % (y.index[kd + 1].strftime('%m.%Y'), td, steps[:kd].mean(),
steps[kd:].mean()))
perm = np.array([abs(scan(rng.permutation(steps))[1]) for _ in range(2000)])
print('перестановочний p для зміни темпу = %.3f (95-й процентиль max|t| = %.2f)' %
(np.mean(perm >= abs(td)), np.percentile(perm, 95)))


fig, ax = plt.subplots(figsize=(11, 3.8))
ax.plot(y.index, values, color='#2C6E9B', lw=2)
ax.axvline(y.index[k], color='#C0392B', ls='--', label='скан рівня: %s' %
y.index[k].strftime('%m.%Y'))
ax.hlines([values[:k].mean(), values[k:].mean()], [y.index[0], y.index[k]],
[y.index[k - 1], y.index[-1]],
          colors='#C0392B', lw=2)
ax.axvline(y.index[kd + 1], color='#27AE60', ls=':', label='скан приростів: %s' %
y.index[kd + 1].strftime('%m.%Y'))
ax.set_title('Точка зміни режиму')
ax.set_ylabel('грн за літр')
ax.legend()
plt.tight_layout()
plt.show()
```

Результат виконання:

```
точка зміни: 03.2017, t = 16.07
середнє до: 27.85 грн, після: 44.85 грн
бутстреп приростів (2000 рядів без зламу): медіана max|t| = 13.5, 95-й процентиль =
16.1
p-значення зламу проти «просто тренду» = 0.054
скан приростів: швидкість зростання змінилась у 02.2015, t = 4.35, приріст до 0.05,
після 0.74 грн/міс
перестановочний p для зміни темпу = 0.006 (95-й процентиль max|t| = 3.40)
```

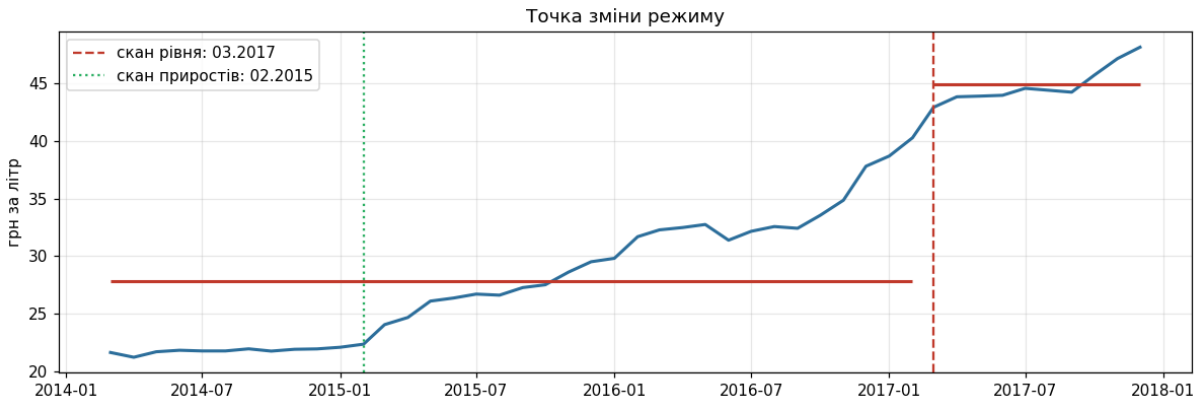


Рисунок 8 — Точка зміни рівня (03.2017) і зміни темпу зростання (02.2015)

Скан рівня знаходить 03.2017: t = 16,07, середнє до — 27,85 грн, після — 44,85 грн. Але в трендовому ряді будь-яке ділення дає велике t, і перебрано 35 дат, тож звичайна t-таблиця непридатна. Бутстреп приростів (2000 рядів без зламу) дає 95-й процентиль max|t| = 16,1 і p = 0,054: **злам рівня не значущий** порівняно зі звичайним трендом. Натомість скан приростів знаходить зміну темпу зростання в 02.2015 (з +0,05 до +0,74 грн за місяць, перестановочний p = 0,006) — це вже значуща зміна режиму з зовнішнім поясненням: перехід до плаваючого курсу гривні та інфляція 2015 року. Від сезонного сплеску вона відрізняється тим, що не повторюється щороку й не повертається назад.

Етап 9. Рецензія на чужий аналіз

```
bad = pd.read_csv('ціни_україна_2014_2017.csv')
bad['date'] = pd.to_datetime(dict(year=bad['рік'], month=bad['місяць'], day=1))
pick = lambda market, good: bad[(bad['ринок'] == market) & (bad['товар'] ==
good)].set_index('date')['ціна'].sort_index()
vb = pick('Vinnytsia', 'Buckwheat grits - Retail')
beef = pick('Vinnytsia', 'Meat (beef) - Retail')
lv = pick('Lviv', 'Buckwheat grits - Retail')


print('клітинка 3: спостережень %d, а місяців у сітці %d; пропущено: %s'
      % (len(vb), len(grid), [d.strftime('%m.%Y') for d in
grid.difference(vb.index)]))
for name, x in [('гречка', vb), ('яловичина', beef)]:
    mae = x.diff().abs().mean()
    print('клітинка 7: %-9s MAE наївного = %.2f грн, середня ціна = %.2f грн,
MAE/ціна = %.1f %%'
          % (name, mae, x.mean(), 100 * mae / x.mean()))
print('клітинка 9: кореляція рівнів = %.3f, кореляція місячних змін = %.3f'
      % (vb.corr(lv), vb.diff().corr(lv.diff())))
p_adf = adfuller(vb, autolag='AIC')[1]
print('клітинка 13: ADF p = %.3f > 0.05 → H0 «нестаціонарний» НЕ відхиляється' %
p_adf)
vbg = vb.reindex(grid).interpolate('linear')
st = STL(vbg, period=12, robust=True).fit()
print('клітинка 11: F_S гречки = %.3f' % max(0, 1 - st.resid.var() / (st.seasonal +
st.resid).var()))
z = (vb - vb.mean()) / vb.std()
```

```
print('клітинка 15: поріг у коді 1.3σ (а в тексті 2σ); |z| > 2 мають %d місяців, |z|
> 1.3 — %d'
      % (int((z.abs() > 2).sum()), int((z.abs() > 1.3).sum())))
print('клітинка 19: останнє значення %.2f грн, а прогноз на 12.2018 = 8.95 грн — це
падіння, хоча висновок «зростання»'
      % vb.iloc[-1])
```

Результат виконання:

```
клітинка 3: спостережень 45, а місяців у сітці 46; пропущено: ['08.2017']
клітинка 7: гречка    MAE наївного = 1.38 грн, середня ціна = 21.32 грн, MAE/ціна =
6.5 %
клітинка 7: яловичина MAE наївного = 1.51 грн, середня ціна = 78.47 грн, MAE/ціна =
1.9 %
клітинка 9: кореляція рівнів = 0.996, кореляція місячних змін = 0.936
клітинка 13: ADF p = 0.233 > 0.05 → H0 «нестаціонарний» НЕ відхиляється
клітинка 11: F_S гречки = 0.262
клітинка 15: поріг у коді 1.3σ (а в тексті 2σ); |z| > 2 мають 0 місяців, |z| > 1.3 —
12
клітинка 19: останнє значення 17.67 грн, а прогноз на 12.2018 = 8.95 грн — це
падіння, хоча висновок «зростання»
```

Таблиця 7 — Методологічні помилки у звіті колеги

Клітинка	Помилка	Чим небезпечна
3	«пропусків немає» — перевірено ряд без повної сітки (45 точок замість 46)	лаги й сезонний період зсуваються на місяць
5	висновок про купівельну спроможність за номінальною ціною і двома точками	висновок, який дані не підтримують
7	MAE різних товарів порівняно в гривнях	масштаб різний; відносно ціни висновок протилежний (1,9 % проти 6,5 %)
9	кореляція рівнів трендових рядів подана як причинність	спільний тренд дає хибну «залежність»
11	«STL виділив сезонність, отже ряд сезонний»	F_S гречки = 0,26 < 0,3 — сезонні моделі без підстав
13	ADF прочитано навпаки, потім подвійне диференціювання без KPSS	неправильний порядок d
15	«аномалії» на рівні трендового ряду з порогом 1,3σ замість заявлених 2σ	розслідування подій, яких не було
17	випадковий train_test_split, порівняння з np.sort(y_te)	витік майбутнього і метрика без сенсу
19	фінальна модель ARIMA(2, 2, 1) не та, що оцінювалась; прогноз 8,95 грн суперечить висновку «зростання»	неперевірена модель, хибний висновок
21	стовпчики з обрізаною віссю Y і 6 вибраних місяців	маніпулятивна візуалізація
22	загальні висновки повторюють хибні твердження	помилки потрапляють у підсумок

Три найсерйозніші помилки виправляються так: привести ряд до повної місячної сітки й позначити заповнений місяць; замість випадкового поділу оцінювати прогноз рухомим початком і порівнювати з наївним базисом на тих

самих фолдах; читати ADF правильно (велике p — нестаціонарний), перевіряти разом із KPSS і прогнозувати тією моделлю, яку оцінювали.

Етап 10. Експорт ряду для Orange

```
export = pd.DataFrame({'date': y.index.strftime('%Y-%m-%d'), 'price': y.values})
export.to_csv('ряд_для_orange.csv', index=False, encoding='utf-8')


check = pd.read_csv('ряд_для_orange.csv')
print('рядків даних:', len(check), '| порожніх значень:',
int(check.isna().sum().sum()))
print('стовпці:', list(check.columns), '| перша дата:', check['date'].iloc[0], '|
остання:', check['date'].iloc[-1])
print('дати у форматі РРРР-ММ-ДД:', bool(check['date'].str.fullmatch(r'\d{4}-\d{2}-
01').all()))
print('збігається з рядом у Python:', bool(np.allclose(check['price'].values,
y.values)))
print(check.iloc[40:43].to_string(index=False))
```

Результат виконання:

```
рядків даних: 46 | порожніх значень: 0
стовпці: ['date', 'price'] | перша дата: 2014-03-01 | остання: 2017-12-01
дати у форматі РРРР-ММ-ДД: True
збігається з рядом у Python: True
      date    price
2017-07-01 44.54000
2017-08-01 44.37335
2017-09-01 44.20670
```

Файл ряд_для_orange.csv має 46 рядків даних і жодного порожнього значення; дата записана як РРРР-ММ-ДД, значення збігаються з рядом у Python.

Етап 11. Потік опрацювання даних в Orange

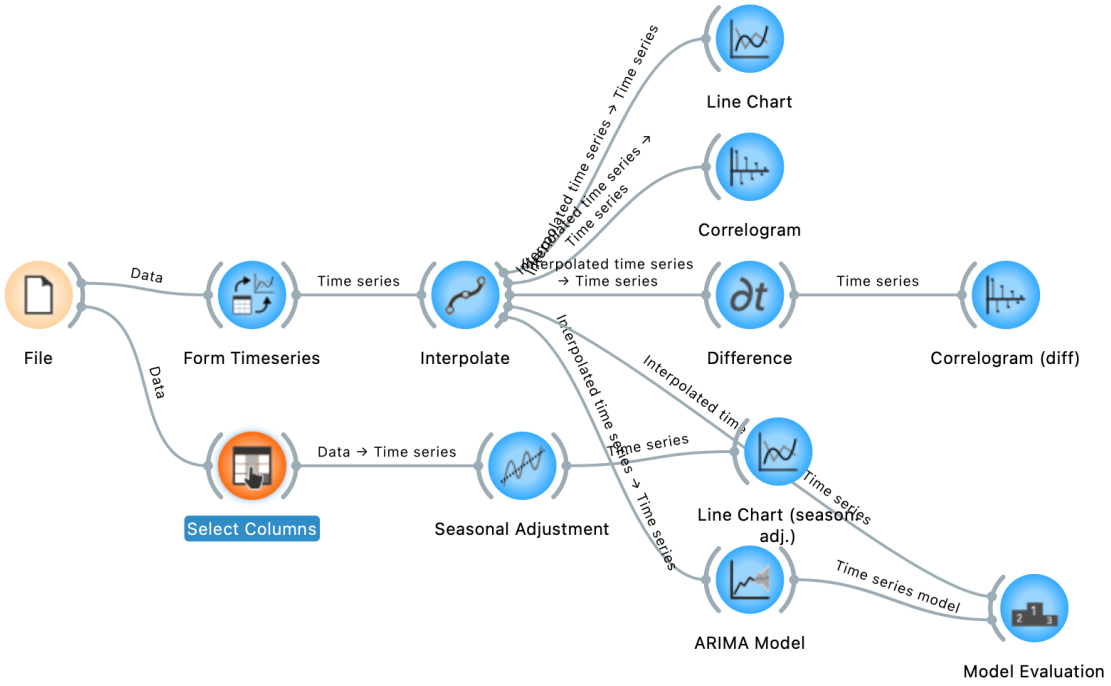


Рисунок 9 — Потік Orange (потік_orange.ows)

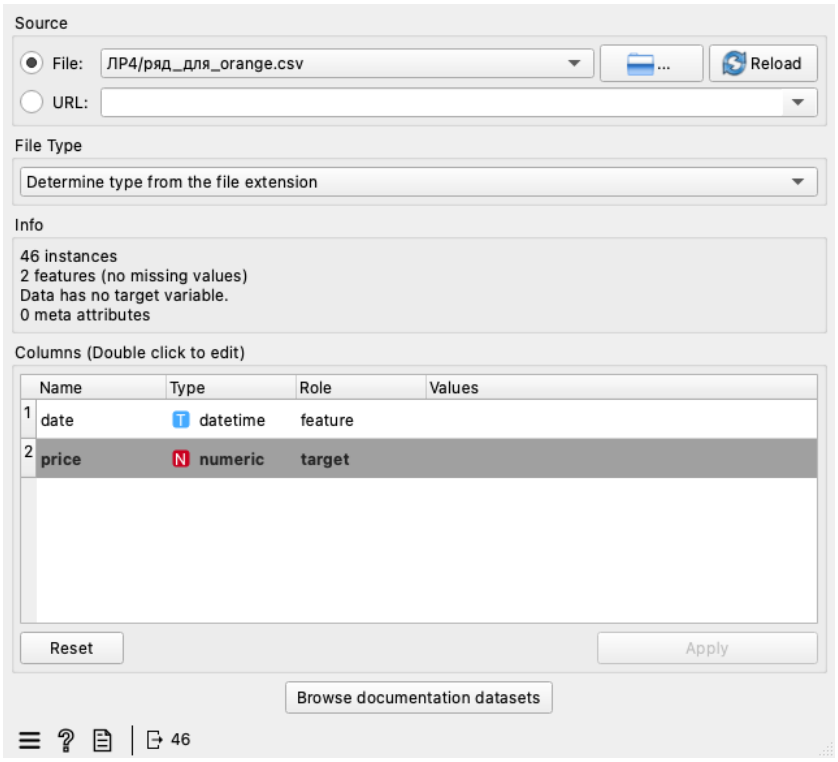


Рисунок 10 — Віджет File: date — datetime, price — numeric, роль target

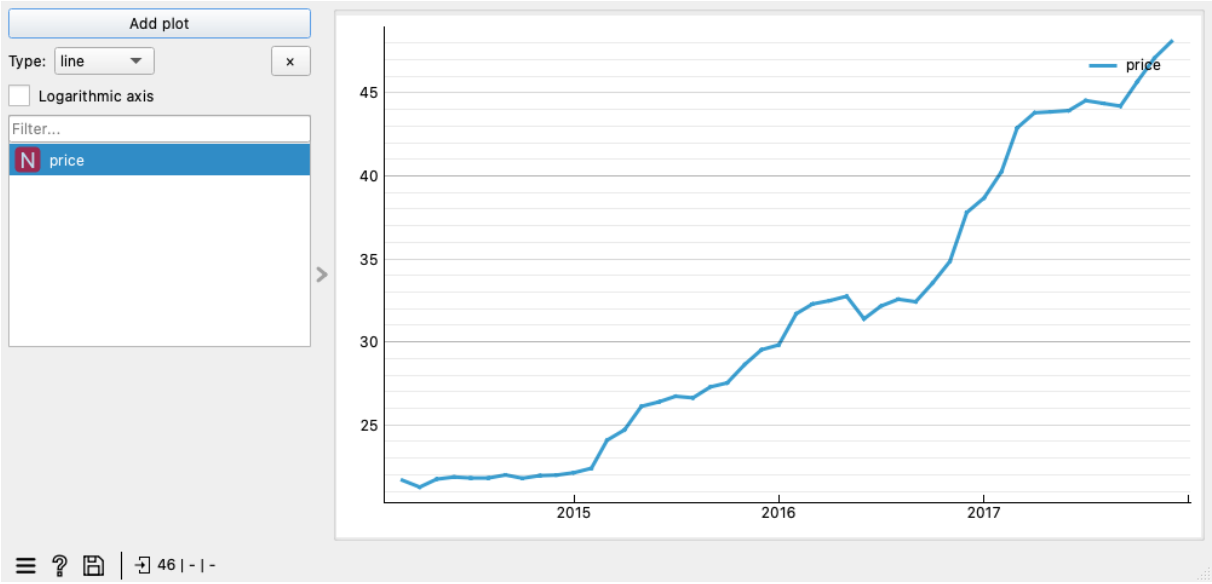


Рисунок 11 — Line Chart в Orange

Потік File → Form Timeseries → Interpolate → Line Chart побудовано в Orange 3.40 з додатком Orange3-Timeseries 0.6.3, де віджет As Timeseries перейменовано на Form Timeseries. Віджет File розпізнав date як datetime, a price — як numeric; price довелося зробити цільовою змінною (Role = target), бо без цього ARIMA Model видає «Input time series doesn't contain a target variable». Line Chart в Orange збігається з рис. 1.

Етап 12. Автокореляція та диференціювання

```
display(Image(SHOTS + 'o12_correlogram.png', width=800))
display(Image(SHOTS + 'o12_correlogram_diff.png', width=800))


from scipy.signal import argrelextrema


def orange_like(x):
    r = acf(x, nlags=int(0.9 * len(x)), fft=True)
    lags = argrelextrema(np.abs(r), np.greater, order=3)[0]
    band = 1.96 * np.sqrt((1 + 2 * (r[lags] ** 2).sum()) / len(x))
    return pd.Series(r[lags], index=lags).round(3), round(band, 3)


lvl, band_lvl = orange_like(y.values)
dif, band_dif = orange_like(y.diff().interpolate(limit_direction='both').values)
print('Orange показує лише локальні максимуми |ACF| (argrelextrema, order=3) до лагу
0.9·n:')
print('  рівень   :', lvl.to_dict(), '| смуга ±%.3f' % band_lvl)
print('  різниця  :', dif.to_dict(), '| смуга ±%.3f' % band_dif)
print('statsmodels на тих самих лагах: рівень', np.round(acf(y,
nlags=41)[lvl.index], 3),
      '| різниця', np.round(acf(y.diff().interpolate(limit_direction='both'),
nlags=41)[dif.index], 3))


print()
print('Seasonal Adjustment напряму з Interpolate — помилка Orange3-Timeseries 0.6.3
«variable date is not in domain»:')
```

```
display(Image(SHOTS + 'o12_seasonal_adjustment_error.png', width=260))
print('Обхід: File → Select Columns (date прибрано) → Seasonal Adjustment (additive,
період 12) → Line Chart')
display(Image(SHOTS + 'o12_season_adj_line_chart.png', width=900))


from statsmodels.tsa.seasonal import seasonal_decompose
classic = seasonal_decompose(y.values, model='additive', period=12)
compare_season = pd.DataFrame({'Orange (класична, additive)': classic.seasonal[:12],
'STL': stl.seasonal.values[:12]},
                              index=[d.strftime('%m.%Y') for d in y.index[:12]])
display(compare_season.round(2).T)
print('амплітуда сезонної складової: класична %.2f грн, STL %.2f грн'
      % (classic.seasonal.max() - classic.seasonal.min(), stl.seasonal.max() -
stl.seasonal.min()))
print('кореляція двох сезонних складових: %.2f' % np.corrcoef(classic.seasonal,
stl.seasonal)[0, 1])
```

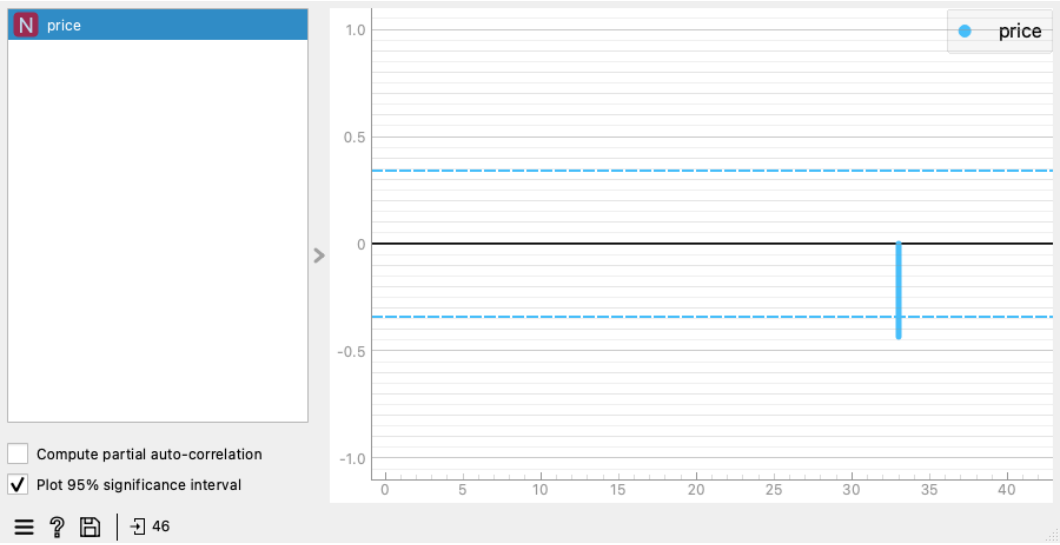


Рисунок 12 — Correlogram рівня в Orange

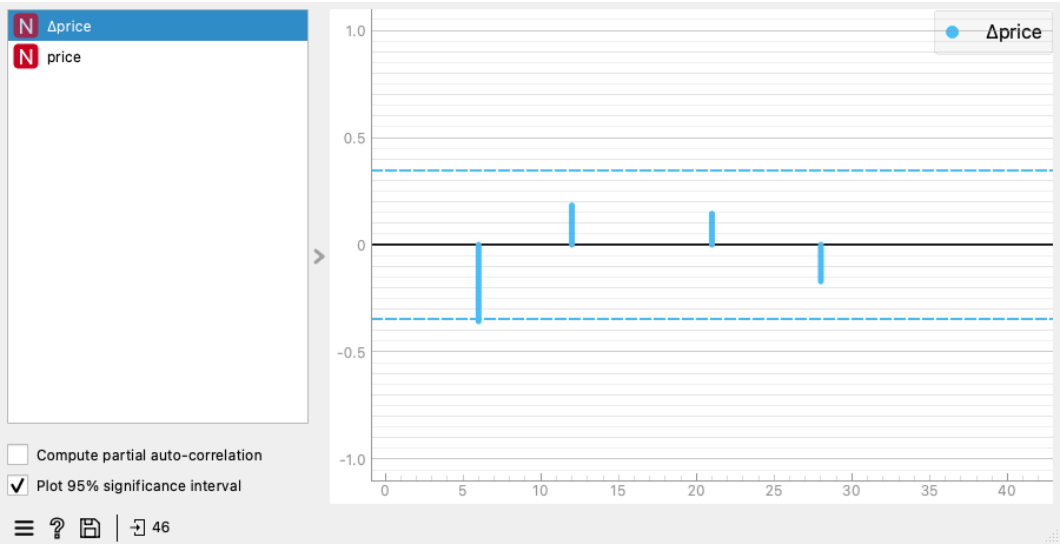


Рисунок 13 — Correlogram першої різниці в Orange

Результат виконання:

```
Orange показує лише локальні максимуми |ACF| (argrelextrema, order=3) до лагу 0.9·n:
  рівень    : {33: -0.434} | смуга ±0.339
```

```
  різниця  : {6: -0.357, 12: 0.184, 21: 0.145, 28: -0.171} | смуга ±0.345
statsmodels на тих самих лагах: рівень [-0.434] | різниця [-0.357  0.184  0.145 -
0.171]


Seasonal Adjustment напряму з Interpolate — помилка Orange3-Timeseries 0.6.3
«variable date is not in domain»:
Обхід: File → Select Columns (date прибрано) → Seasonal Adjustment (additive, період
12) → Line Chart
амплітуда сезонної складової: класична 2.36 грн, STL 4.92 грн
кореляція двох сезонних складових: 0.82
```

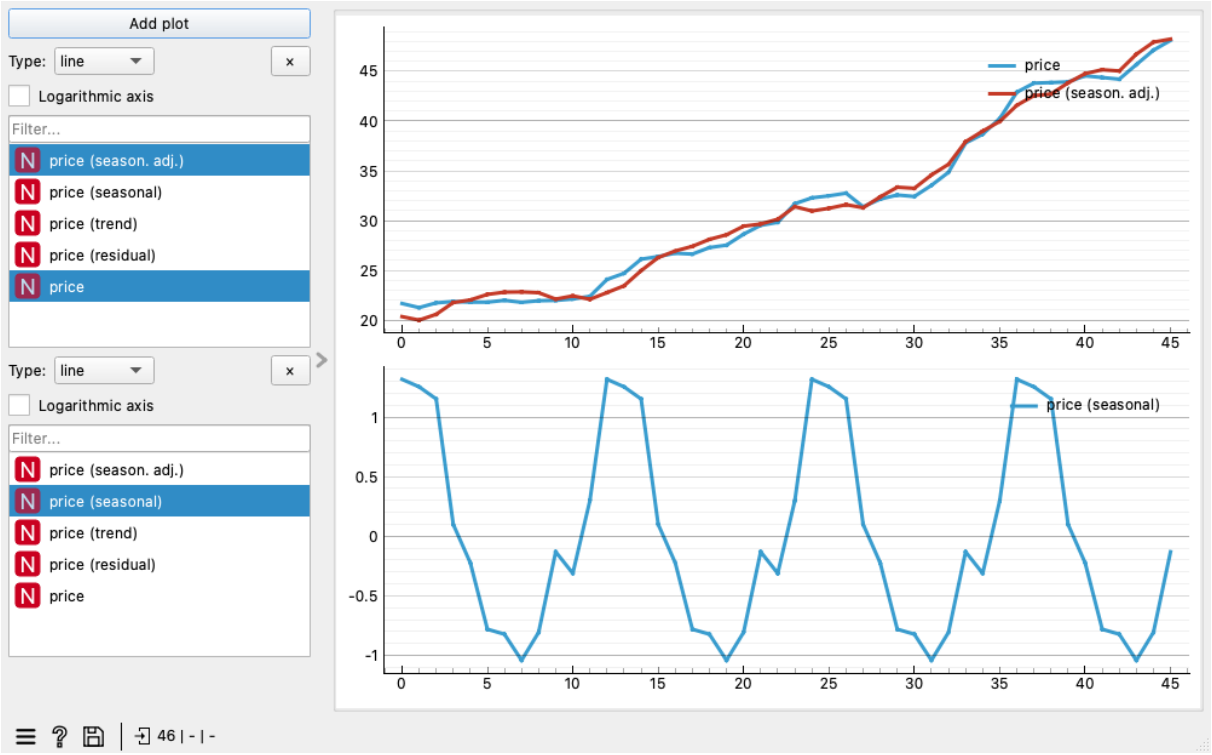


Рисунок 14 — Seasonal Adjustment в Orange: ряд і сезонно скоригований ряд, сезонна складова

Orange рахує ту саму автокореляцію, що й statsmodels, але показує лише лаги з локальним максимумом |ACF| (до лагу 0,9·n = 41) і одну смугу значущості для всіх лагів. Для рівня це один стовпчик на лагу 33, а для різниці — лаги 6, 12, 21 і 28; значення на цих лагах збігаються з Python до третього знака. Значущий від'ємний лаг 6 різниці видно в обох середовищах, а лаг 1 Orange не показує.

Seasonal Adjustment напряму після Interpolate в Orange3-Timeseries 0.6.3 завершується помилкою «variable date is not in domain» — віджет відкидає часову змінну, але лишає посилання на неї. Обхід: File → Select Columns (прибрати date) → Seasonal Adjustment → Line Chart. Orange використовує класичне розкладання ковзними середніми: його сезонна складова має

амплітуду 2,36 грн проти 4,92 грн у STL, але той самий рисунок (кореляція 0,82).

Етап 13. Модель і оцінювання в Orange

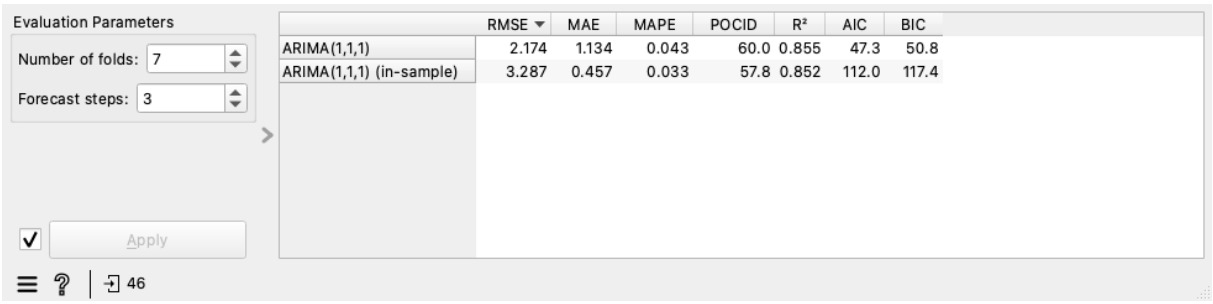


Рисунок 15 — Model Evaluation: 7 фолдів × 3 кроки



Рисунок 16 — Model Evaluation з типовими 20 фолдами

```
display(Image(SHOTS + 'o13_arima.png', width=360))
print('Model Evaluation, 7 фолдів × 3 кроки:')
display(Image(SHOTS + 'o13_model_evaluation_7x3.png', width=900))
print('Model Evaluation з типовими налаштуваннями (20 фолдів × 3 кроки):')
display(Image(SHOTS + 'o13_model_evaluation_20x3_error.png', width=900))


max_lag = max((1, 1, 1))
for n_folds in (20, 7):
    need = n_folds * H + max_lag
    print('%2d фолдів: потрібно %d × %d + %d = %d спостережень, є %d → %s'
          % (n_folds, n_folds, H, max_lag, need, n, 'відмова' if need > n else
'рахує'))
print('найбільше фолдів при 3 кроках: %d' % ((n - max_lag) // H))
print()
print('фолди Orange (train_end = −fold·steps):')
for f in range(1, 8):
    end = n - 3 * f
    print('  фолд %d: навчання на 03.2014–%s, прогноз %s–%s'
          % (f, y.index[end - 1].strftime('%m.%Y'), y.index[end].strftime('%m.%Y'),
y.index[end + 2].strftime('%m.%Y')))
```

Результат виконання:

```
Model Evaluation, 7 фолдів × 3 кроки:
Model Evaluation з типовими налаштуваннями (20 фолдів × 3 кроки):
20 фолдів: потрібно 20 × 3 + 1 = 61 спостережень, є 46 → відмова
 7 фолдів: потрібно 7 × 3 + 1 = 22 спостережень, є 46 → рахує
найбільше фолдів при 3 кроках: 15


фолди Orange (train_end = −fold·steps):
```

```
  фолд 1: навчання на 03.2014–09.2017, прогноз 10.2017–12.2017
  фолд 2: навчання на 03.2014–06.2017, прогноз 07.2017–09.2017
  фолд 3: навчання на 03.2014–03.2017, прогноз 04.2017–06.2017
  фолд 4: навчання на 03.2014–12.2016, прогноз 01.2017–03.2017
  фолд 5: навчання на 03.2014–09.2016, прогноз 10.2016–12.2016
  фолд 6: навчання на 03.2014–06.2016, прогноз 07.2016–09.2016
  фолд 7: навчання на 03.2014–03.2016, прогноз 04.2016–06.2016
```

Таблиця Model Evaluation для ARIMA(1, 1, 1): RMSE 2,174, MAE 1,134, MAPE 0,043, POCID 60,0. З типовими 20 фолдами Orange відмовляється рахувати: перевірка n_folds × forecast_steps + max_lag ≤ довжина ряду дає 20·3 + 1 = 61 > 46. Найбільше можна 15 фолдів. Сім фолдів Orange перевіряють ті самі 21 останні місяці (04.2016–12.2017), що й у Python.

Етап 14. Чому числа не збігаються

```
orange = {'RMSE': 2.174, 'MAE': 1.134, 'MAPE': 0.043, 'POCID': 60.0}

e = arima_true - arima_pred
mine = {
    'RMSE': np.sqrt(np.sum(e ** 2) / len(e)),
    'MAE': np.median(np.abs(e)),
    'MAPE': np.mean(np.abs(e)) / np.mean(np.abs(arima_true)),
    'POCID': 100 * np.mean(np.diff(arima_true) * np.diff(arima_pred) > 0),
}
formulas = {
    'RMSE': 'sqrt(Σe² / n)',
    'MAE': 'median |e|',
    'MAPE': 'mean|e| / mean|y|  (частка)',
    'POCID': '100 · mean[Δy · Δŷ > 0] по склеєних фолдах',
}
usual = {
    'RMSE': np.sqrt(np.mean(e ** 2)),
    'MAE': np.mean(np.abs(e)),
    'MAPE': np.mean(np.abs(e / arima_true)) * 100,
    'POCID': np.nan,
}
rep = pd.DataFrame({'Orange': orange, 'відтворення в Python': mine, 'формула
Orange': formulas,
                    'звична формула (Частина 1)': usual})
rep['збіг до 3-го знака'] = [round(mine[k], 3) == orange[k] for k in rep.index]
display(rep.round(4))
print('прогнози ARIMA(1,1,1) по 7 фолдах (порядок Orange: фолд 1 — останні 3
місяці):')
print(np.round(arima_pred, 3))
```

Таблиця 8 — Метрики Orange і їхнє відтворення в Python

Метрика	Orange	Відтворення	Формула Orange	Звична формула
RMSE	2,174	2,1736	sqrt(Σe² / n)	2,1736
MAE	1,134	1,1342	median |e|	1,7099
MAPE	0,043	0,0434	mean|e| / mean|y| (частка)	4,4020
POCID	60,000	60,0000	100 · mean[Δy · Δŷ > 0] по склеєних фолдах	—

Усі чотири числа Orange відтворено до третього знака, бо прогнози в обох середовищах однакові. Назві не відповідають MAE (у Orange це медіана |e| = 1,134, а не середнє 1,710) і MAPE (mean|e| / mean|y| = 0,043 — частка, а не відсоток, і інша формула, ніж mean|e / y|·100 = 4,40 %). Одразу збігся RMSE (2,174), бо його формула однакова. Перш ніж порівнювати числа двох інструментів, треба перевірити дані, модель, схему перевірки і насамперед формули та одиниці метрик.

Етап 15. Матриця напрямку зміни

```
dt, dp = np.diff(arima_true), np.diff(arima_pred)
actual = np.where(dt > 0, 'зросла', 'впала')
predicted = np.where(dp > 0, 'зросла', 'впала')
matrix = pd.crosstab(pd.Series(actual, name='насправді'), pd.Series(predicted,
name='модель'))
display(matrix)
hit = np.mean(actual == predicted) * 100
print('частка правильних напрямків: %.1f %% (POCID Orange = %.1f %%)' % (hit,
orange['POCID']))
boundary = np.arange(1, len(dt) + 1) % H == 0
print('пар усього: %d, із них на стику фолдів: %d' % (len(dt), int(boundary.sum())))
print('правильних напрямків усередині фолдів: %.1f %%' % (np.mean((actual ==
predicted)[~boundary]) * 100))
always_up = np.mean(actual == 'зросла') * 100
print('тривіальне правило «завжди зростає»: %.1f %%' % always_up)
```

Таблиця 9 — Матриця напрямку зміни (20 пар)

Насправді \ модель	впала	зросла
впала	4	5
зросла	3	8

Результат виконання:

```
частка правильних напрямків: 60.0 % (POCID Orange = 60.0 %)
пар усього: 20, із них на стику фолдів: 6
правильних напрямків усередині фолдів: 57.1 %
тривіальне правило «завжди зростає»: 55.0 %
```

Правильних напрямків 12 з 20 — 60 %, що збігається з POCID Orange. Шість пар припадають на стик фолдів, бо Orange склеює фолди; усередині фолдів правильних 57,1 %. Для рішення «купувати зараз чи чекати» модель майже марна: правило «ціна завжди зростає» вгадує 55 % пар. MAE і RMSE відповідають на інше питання — наскільки далеко прогноз від ціни, а не чи вгадано знак зміни.

ВИСНОВКИ

Ряд роздрібних цін на сметану в Херсоні має 46 місяців і один пропуск — серпень 2017, якого немає в усьому наборі; його заповнено лінійною інтерполяцією (44,37 грн) однаково в Python і Orange.

Сезонність є (F_S = 0,82, робастно 0,53 > 0,3): ціна росте взимку–навесні й стоїть влітку–восени. Але тренд значно сильніший (F_T = 0,997), рівень нестаціонарний, а перша різниця стаціонарна — тести ADF і KPSS узгоджуються.

Найкращим методом прогнозу виявився найпростіший дрейф (MAE 1,17, RMSE 1,63). ARIMA(1, 1, 1) без константи йому програла (1,71 і 2,17), а ARIMA(0, 1, 1) з дрейфом лише зрівнялась — складна модель простого базису не перевершила.

Злам рівня 03.2017 незначущий проти тренду (p = 0,054), а зміна темпу зростання в 02.2015 значуща (p = 0,006) і пояснюється девальвацією гривні.

У рецензованому звіті знайдено 11 методологічних помилок; найнебезпечніші — неповна сітка, випадковий поділ часового ряду і перевернутий висновок тесту ADF.

В Orange та сама модель на тих самих 7 фолдах дала ті самі прогнози. Метрики відрізнились лише через означення: «MAE» там — медіана |e|, «MAPE» — частка mean|e| / mean|y|; RMSE збігся. Усі чотири числа Orange відтворено в Python до третього знака.

Orange3-Timeseries 0.6.3 має дві практичні пастки: для ARIMA і Model Evaluation ціну треба явно зробити target, а Seasonal Adjustment не працює з часовою змінною й потребує обходу через Select Columns. При 20 фолдах Orange відмовляє, бо 20·3 + 1 = 61 > 46.

Через 46 спостережень не вдалося надійно оцінити сезонну модель, використати більше 15 фолдів чи довший горизонт, а POCID 60 % проти 55 % для правила «завжди зростає» на 20 парах статистично не розрізняються.